```
!pip install semantic-kernel google-generativeai -q
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 74.7/74.7 kB 3.5 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 88.5/88.5 kB 6.9 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.5/46.5 kB 3.9 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 893.2/893.2 kB 33.1 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 92.9/92.9 kB 9.8 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 217.9/217.9 kB 21.9 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 126.7/126.7 kB 13.0 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 191.3/191.3 kB 19.3 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 55.8/55.8 kB 5.6 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 106.6/106.6 kB 11.4 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 35.5/35.5 MB 31.3 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 211.8/211.8 kB 21.5 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 412.9/412.9 kB 37.4 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.5/4.5 MB 126.0 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 117.0/117.0 kB 10.9 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.2/2.2 MB 95.7 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 57.3/57.3 kB 5.0 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 119.7/119.7 kB 13.1 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 224.4/224.4 kB 21.8 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 331.1/331.1 kB 30.2 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 71.5/71.5 kB 6.7 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 753.1/753.1 kB 56.5 MB/s eta 0:00:00
  Building wheel for pybars4 (setup.py) ... done
  Building wheel for PyMeta3 (setup.py) ... done
ERROR: pip's dependency resolver does not currently take into account all the pa
pydrive2 1.21.3 requires cryptography<44, but you have cryptography 46.0.2 which
pydrive2 1.21.3 requires pyOpenSSL<=24.2.1,>=19.1.0, but you have pyopenssl 25.3
```

```
import google.generativeai as genai
import semantic_kernel as sk
from semantic_kernel.functions.kernel_plugin import KernelPlugin
from semantic_kernel.functions import kernel_function
import asyncio
```

```
API_KEY = "AIzaSyDvmXzyjXOHipKFddCzIUXpCWBFIb68fH0"  # replace with your key
genai.configure(api_key=API_KEY)
model_name = "gemini-2.5-flash"
```

```
kernel = sk.Kernel()
```

```
@kernel_function(
    description="Summarizes a given text using Gemini 2.5 Flash",
    name="Summarize",
)
def summarize_text(text: str) -> str:
    model = genai.GenerativeModel(model_name)
    response = model.generate_content(f"Summarize this in one concise paragraph
    return response.text.strip()
```

```
summarization_plugin = KernelPlugin(
    name="SummarizationPlugin", description="Summarizes text using Gemini"
)
kernel.add_plugin(summarization_plugin)
kernel.add_function(plugin_name="SummarizationPlugin", function=summarize_text)
```

```
KernelFunctionFromMethod(metadata=KernelFunctionMetadata(name='Summarize',
plugin_name='SummarizationPlugin', description='Summarizes a given text using
Gemini 2.5 Flash', parameters=[KernelParameterMetadata(name='text',
description=None, default_value=None, type_='str', is_required=True,
type_object=<class 'str'>, schema_data={'type': 'string'},
include_in_function_choices=True)], is_prompt=False, is_asynchronous=False,
return_parameter=KernelParameterMetadata(name='return', description='',
default_value=None, type_='str', is_required=True, type_object=<class 'str'>,
schema_data={'type': 'string'}, include_in_function_choices=True),
additional_properties={}), invocation_duration_histogram=
<opentelemetry.metrics._internal.instrument._ProxyHistogram object at
0x7ab6bdc74950>, streaming_duration_histogram=
<opentelemetry.metrics._internal.instrument._ProxyHistogram object at
0x7ab6bdc74920>, method=<function summarize_text at 0x7ab6bdc6d3a0>,
stream_method=None)
```

```
@kernel_function(
    description="Analyzes the sentiment of text",
    name="AnalyzeSentiment",
)
def sentiment_score(text: str) -> str:
    positive = ["good", "great", "excellent", "amazing", "positive"]
    negative = ["bad", "poor", "terrible", "negative", "worst"]

    pos = sum(word in text.lower() for word in positive)
    neg = sum(word in text.lower() for word in negative)

    if pos > neg:
        return "😊 Positive sentiment"
    elif neg > pos:
        return "😞 Negative sentiment"
    else:
        return "😐 Neutral sentiment"

sentiment_plugin = KernelPlugin(
    name="SentimentSkill", description="Analyzes sentiment"
)
kernel.add_plugin(sentiment_plugin)
kernel.add_function(plugin_name="SentimentSkill", function=sentiment_score)
```

```
KernelFunctionFromMethod(metadata=KernelFunctionMetadata(name='AnalyzeSentiment'
plugin_name='SentimentSkill', description='Analyzes the sentiment of text',
parameters=[KernelParameterMetadata(name='text', description=None,
default_value=None, type_='str', is_required=True, type_object=<class 'str'>,
schema_data={'type': 'string'}, include_in_function_choices=True)],
is_prompt=False, is_asynchronous=False,
return_parameter=KernelParameterMetadata(name='return', description='',
default_value=None, type_='str', is_required=True, type_object=<class 'str'>,
schema_data={'type': 'string'}, include_in_function_choices=True),
```

```
additional_properties={}), invocation_duration_histogram=
<opentelemetry.metrics._internal.instrument._ProxyHistogram object at
0x7ab6bdc758b0>, streaming_duration_histogram=
<opentelemetry.metrics._internal.instrument._ProxyHistogram object at
0x7ab6bdc74b00>, method=<function sentiment_score at 0x7ab6bdc6ce00>,
stream_method=None)
```

```
async def composed_skill(input_text: str):
    summary = summarize_text(input_text)
    print("\n 🔹 Summary:\n", summary)

    sentiment = sentiment_score(summary)
    print("\n 🔹 Sentiment:\n", sentiment)
```

```
input_text = """
I recently tried the new restaurant downtown. The food was excellent,
and the service was great. The ambiance made me feel very comfortable.
However, the waiting time was a bit long, but overall it was a good experience.
"""

await composed_skill(input_text)
```

```
🔹 Summary:
This new downtown restaurant offers excellent food, great service, and a comfor

🔹 Sentiment:
😊 Positive sentiment
```